



Register Number															
--------------------	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

SRM Institute of Science and Technology
College of Engineering and Technology
School of Computing

Set - D

SRM Nagar, Kattankulathur – 603203, Chengalpattu District, Tamil Nadu
Academic Year: 2025-26 (EVEN SEM)

Test: LLT & FT- IV

Date: 29.04.2026

Course Code & Title:

Duration: 100 Minutes

Year & Sem: III Year / VI Sem

Max. Marks: 50

PART A—MCQ

Q.No Correct Option Answer

- | | | |
|----|---|--|
| 1 | b | Encoding |
| 2 | b | Dropping duplicates |
| 3 | c | concat() |
| 4 | b | Convert data into suitable format for analysis |
| 5 | b | Data aggregation |
| 6 | b | The elevation and azimuthal angles of the camera viewing the 3D plot |
| 7 | b | plt.colorbar() |
| 8 | a | plt.text(..., bbox=dict(facecolor='red', alpha=0.5)) |
| 9 | a | sns.violinplot() |
| 10 | b | annot=True |

Part – B (5 x 5 = 25 Marks)	
Instructions: Answer ANY 5 questions	
Q. No	Questions
11	A dataset contains employee salary details collected from different departments, where salaries are in different ranges (e.g., ₹10,000–₹50,000 and ₹1,00,000–₹5,00,000). This makes comparison difficult. The analyst needs to standardize the salary values for analysis. a) Define Standardization (1 Mark) b) Explain its purpose (1 Mark) c) Apply standardization using suitable formula interpret how it improves comparison across departments. (2 Marks) d) Display the output (1 Mark) Answer: a) Definition (1 Mark) Standardization is the process of converting data into a common scale with mean = 0 and standard deviation = 1. b) Purpose (1 Mark) • Enables fair comparison of values • Removes scale differences c) Formula & Code (2 Marks) <code>import numpy as np</code> <code>salary = np.array([10000, 20000, 30000, 100000, 200000])</code> <code>mean = np.mean(salary)</code> <code>std = np.std(salary)</code> <code>z = (salary - mean) / std</code> <code>print(z)</code> Interpretation: Standardization removes scale differences by converting salaries into z-scores, enabling fair comparison across departments. It helps identify whether an employee's salary is relatively high or low within their group, independent of the original salary range. d) Final Output (1 Mark) <code>[-0.82 -0.65 -0.48 0.54 1.41]</code>
12	A company maintains two datasets: one dataset contains customer details (Customer_ID, Name), and another dataset contains purchase details (Customer_ID, Product). Some customers appear in both datasets, leading to overlapping records. The analyst needs to combine them appropriately. a) Create two datasets representing customer details and purchase details. (1 Mark) b) Combine datasets using appropriate data merging technique. (2 Marks) c) Explain how overlapping data are handled during the merging process. (1 Mark) d) Present the final merged dataset in a suitable format. (1 Mark) Answer: a) Create Dataset (1 Mark) <code>import pandas as pd</code> <code>df1 = pd.DataFrame({</code> <code> "Customer_ID": [1, 2, 3],</code> <code> "Name": ["A", "B", "C"]</code> <code>})</code> <code>df2 = pd.DataFrame({</code>

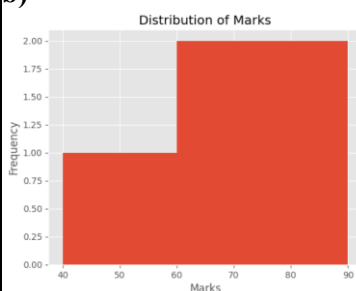
	<pre>"Customer_ID": [2, 3, 4], "Product": ["X", "Y", "Z"] }) b) Combine (2 Marks) merged = pd.merge(df1, df2, on="Customer_ID", how="inner") print(merged) c) Overlapping Handling (1 Mark) • Only common Customer_IDs are included (2, 3) d) Final Output (1 Mark) Customer_ID Name Product 0 2 B X 1 3 C Y</pre>															
13	A dataset contains student marks in different subjects stored as raw scores. The management wants to transform these marks into grades (A, B, C, etc.) for better interpretation. The analyst needs to perform data transformation. a) Create a dataset with student marks. (1 Mark) b) Apply a suitable data transformation technique to convert marks into grades. (2 Marks) c) Explain the purpose of performing this transformation. (1 Mark) d) Present the transformed dataset in a clear format. (1 Mark) Answer: a) Dataset (1 Mark) import pandas as pd <pre>marks = [85, 70, 55, 40] df = pd.DataFrame({"Marks": marks})</pre> b) Transformation (2 Marks) def grade(m): if m >= 80: return "A" elif m >= 60: return "B" elif m >= 50: return "C" else: return "D" <pre>df["Grade"] = df["Marks"].apply(grade) print(df)</pre> c) Purpose (1 Mark) Converts raw data into easier-to-understand format d) Final Output (1 Mark) <table><thead><tr><th></th><th>Marks</th><th>Grade</th></tr></thead><tbody><tr><td>0</td><td>85</td><td>A</td></tr><tr><td>1</td><td>70</td><td>B</td></tr><tr><td>2</td><td>55</td><td>C</td></tr><tr><td>3</td><td>40</td><td>D</td></tr></tbody></table>		Marks	Grade	0	85	A	1	70	B	2	55	C	3	40	D
	Marks	Grade														
0	85	A														
1	70	B														
2	55	C														
3	40	D														
14	An academic coordinator wants to analyze the distribution of student marks in a class to understand how marks are spread across different ranges and identify concentration of scores. a) Create a dataset of student marks. (1 Mark) b) Plot a histogram using an appropriate number of bins. (2 Marks) c) Identify the distribution pattern (e.g., concentration or spread) from the histogram. (1 Mark) d) Present the output and summarize the observed distribution. (1 Mark) Answer: a) Code															

```
import matplotlib.pyplot as plt

marks = [40, 50, 60, 70, 80, 90, 75, 65]

plt.hist(marks, bins=5)
plt.xlabel("Marks")
plt.ylabel("Frequency")
plt.title("Distribution of Marks")
plt.show()
```

b)



c) Distribution Pattern:

The histogram shows that most marks are concentrated in the **60–80 range**, indicating moderate performance by most students.

d) Interpretation:

The distribution is fairly balanced with slight concentration in the middle range. This indicates that the majority of students scored average marks, with fewer students at very low or very high ranges.

- 15 An analyst suspects a relationship between two variables in a dataset and aims to examine the nature and strength of their association through visualization.

a) Create a dataset with two variables. (1 Mark)

b) Plot a suitable scatter diagram to represent the relationship between the variables. (2 Marks)

c) Analyze the scatter plot to determine the type of correlation (positive, negative, or no correlation) and describe the observed trend. (1 Mark)

d) Interpret the results and explain what the relationship indicates about the variables. (1 Mark)

Answer:

a)

```
import matplotlib.pyplot as plt
```

```
x = [1, 2, 3, 4, 5]
```

```
y = [10, 20, 30, 40, 50]
```

b) Scatter Plot (2 Marks)

```
plt.scatter(hours, marks)
plt.xlabel("Hours Studied")
plt.ylabel("Marks Scored")
plt.title("Hours vs Marks Relationship")
plt.show()
```

c) Analysis of Correlation (1 Mark)

- The scatter plot shows a **positive correlation**
- As hours studied increases, marks also increase
- Points follow an **upward trend**

d) Interpretation (1 Mark)

- There is a **strong relationship** between studying time and performance
- Increased study hours lead to **higher marks**
- Indicates that effort (study time) positively impacts results

16	A company aims to analyze sales data to identify and highlight peak sales values so that key insights can be easily interpreted by decision-makers. The analyst needs to visualize the data and emphasize significant points for better understanding. a) Create a dataset representing sales values. (1 Mark) b) Plot an appropriate graph to visualize the sales data. (1 Mark) c) Analyze the graph to identify significant points (e.g., peak values) and annotate them appropriately. (2 Marks) d) Interpret the highlighted points and explain their significance in decision-making. (1 Mark) Answer: a) Create Dataset (1 Mark) <pre>import matplotlib.pyplot as plt</pre> # Sample data (Monthly sales) <pre>months = ['Jan', 'Feb', 'Mar', 'Apr', 'May'] sales = [200, 350, 300, 500, 450]</pre> b) Plot Graph (1 Mark) <pre>plt.plot(months, sales, marker='o') plt.xlabel("Months") plt.ylabel("Sales") plt.title("Monthly Sales Analysis")</pre> Appropriate visualization (line graph) c) Identify & Annotate Peak (2 Marks) <pre># Find peak value max_sales = max(sales) max_index = sales.index(max_sales)</pre> # Highlight and annotate peak <pre>plt.scatter(months[max_index], max_sales, color='red') plt.annotate(f'Peak: {max_sales}', (months[max_index], max_sales), textcoords="offset points", xytext=(0,10), ha='center')</pre> plt.show() d) Interpretation (1 Mark) • The highest sales occur in April (500) • This indicates a peak performance period • Helps decision-makers identify: ○ Best-performing time ○ Possible reasons (seasonal demand, promotions) • Can guide future planning and strategy
17	A company wants to examine monthly sales data to compare product categories and understand cumulative trends over time. The analyst must evaluate different visualization techniques—line plot, bar chart, and area plot—to determine their suitability for various analytical purposes. a) Define any two visualization techniques used for representing data trends. (2 Marks) b) Analyze and compare line plot, bar chart, and area plot based on at least three criteria (e.g., purpose, data representation, and interpretability). (2 Marks) c) Evaluate and justify the most appropriate visualization technique for representing cumulative sales trends over time. (1 Mark) Answer: a) Definition of any two visualization techniques (2 Marks) Line Plot: A line plot represents data points connected by lines, used to show trends or changes over time.

Bar Chart:

A bar chart uses rectangular bars to compare values across different categories.

b b) Analysis & Comparison (2 Marks)

Criteria	Line Plot	Bar Chart	Area Plot
Purpose	Shows trends over time	Compares categories	Shows cumulative trends
Data Representation	Continuous data (connected points)	Discrete categories (bars)	Continuous data with filled area
Interpretability	Easy to track changes/trends	Easy to compare individual values	Good for overall magnitude but can overlap
Best Use Case	Time-series analysis	Category comparison	Cumulative contribution over time

c) Evaluation & Justification (1 Mark)

- The area plot is the most appropriate for representing cumulative sales trends over time
- It highlights the total magnitude and contribution of categories
- Provides a clear view of how values accumulate over time

Part – C
(1 x 15= 15 Marks)

Instructions: Answer **ALL** the questions

Q. No	Questions
18	A government organization maintains a large dataset of citizen records that includes names, addresses, and annual income details. The dataset contains inconsistencies in text fields (such as varying cases and extra spaces), and the income values are not categorized, making analysis difficult. The organization wants to clean the text data, classify income into meaningful categories, and summarize the dataset to support policy decisions. The analyst is responsible for preparing the dataset for effective analysis. a) Explain string manipulation techniques used in data cleaning (3 Marks) b) Write Python code to clean text data (removing spaces and standardizing case) (4 Marks) c) Classify income into categories using appropriate method (3 Marks) d) Summarize the dataset using statistical measures (3 Marks) e) Interpret the final output (2 Marks) Answer: a) String Manipulation Techniques (3 Marks) • strip() → Removes leading/trailing spaces • lower() / upper() / title() → Standardizes text case • replace() → Corrects inconsistent formats • split() / join() → Restructures text These techniques help in making text data consistent and clean b) Python Code – Clean Text Data (4 Marks) <pre>import pandas as pd data = { "Name": [" JOHN doe", "alice ", "BoB smith "], "Address": [" chennai ", "DELHI", " mumbai "], "Income": [20000, 50000, 100000] } df = pd.DataFrame(data) # Cleaning text df["Name"] = df["Name"].str.strip().str.title() df["Address"] = df["Address"].str.strip().str.title()</pre>

```
print(df)
```

Output

	Name	Address	Income
0	John Doe	Chennai	20000
1	Alice	Delhi	50000
2	Bob Smith	Mumbai	100000

c) Income Classification (3 Marks)

```
def income_group(x):
```

```
    if x < 30000:
```

```
        return "Low"
```

```
    elif x < 70000:
```

```
        return "Medium"
```

```
    else:
```

```
        return "High"
```

```
df["Category"] = df["Income"].apply(income_group)
```

```
print(df)
```

Output

	Name	Address	Income	Category
0	John Doe	Chennai	20000	Low
1	Alice	Delhi	50000	Medium
2	Bob Smith	Mumbai	100000	High

d) Summarization (3 Marks)

```
summary = df["Income"].describe()
```

```
print(summary)
```

Output

```
count      3
mean    56666.67
std    40414.52
min     20000
max    100000
```

e) Interpretation (2 Marks)

- Text data is now **clean and consistent**
- Income is grouped into **Low, Medium, High categories**
- Majority falls in **Medium/High range**
- Helps in **policy planning and decision-making**

(OR)

19

A financial analyst is working with a dataset containing monthly revenue data of a company with attributes such as Month, Revenue, and Expenses. The analyst has already created basic plots but finds them visually unappealing and difficult to present in reports. To improve the presentation quality, the analyst decides to apply different built-in plot styles available in Matplotlib (such as ggplot, seaborn, classic, etc.) and compare their effectiveness in representing the data. The goal is to select the most suitable style for clear and professional visualization.

- Explain the concept of plot styles in Matplotlib and their importance in data visualization. (3 Marks)
- Write a Python program to create a basic line plot for revenue data. (3 Marks)
- Modify the program to apply different Matplotlib styles (at least three styles) and display the plots. (5 Marks)
- Compare the visual differences between the styles used and discuss their advantages. (2 Marks)
- Identify the most suitable plot style for financial reporting and justify your answer. (2 Marks)

Answer:

a) Concept of Plot Styles & Importance (3 Marks)

Plot Styles

- Predefined **visual themes** in Matplotlib (e.g., ggplot, seaborn, classic)
- Control appearance like **colors, grid, background, fonts**

Importance

- Improves **visual appeal**
- Enhances **readability**
- Makes graphs **professional for reports**
- Helps highlight **trends clearly**

b) Basic Line Plot (3 Marks)

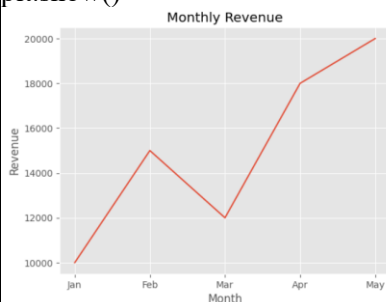
```
import matplotlib.pyplot as plt
```

```
months = ["Jan", "Feb", "Mar", "Apr", "May"]  
revenue = [10000, 15000, 12000, 18000, 20000]
```

```
plt.plot(months, revenue)
```

```
plt.xlabel("Month")  
plt.ylabel("Revenue")  
plt.title("Monthly Revenue")
```

```
plt.show()
```



c) Apply Different Styles (5 Marks)

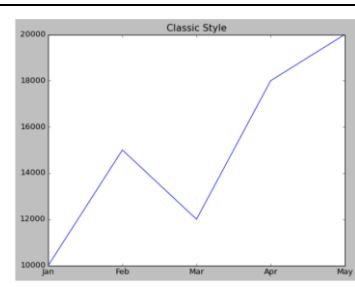
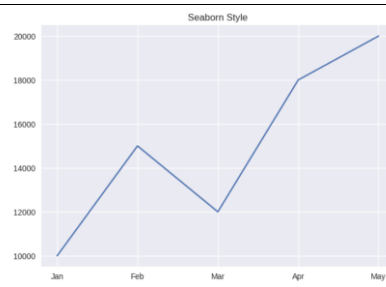
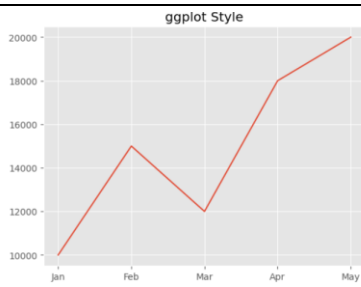
```
import matplotlib.pyplot as plt
```

```
months = ["Jan", "Feb", "Mar", "Apr", "May"]  
revenue = [10000, 15000, 12000, 18000, 20000]
```

```
# Style 1: ggplot  
plt.style.use('ggplot')  
plt.plot(months, revenue)  
plt.title("ggplot Style")  
plt.show()
```

```
# Style 2: seaborn  
plt.style.use('seaborn-v0_8')  
plt.plot(months, revenue)  
plt.title("Seaborn Style")  
plt.show()
```

```
# Style 3: classic  
plt.style.use('classic')  
plt.plot(months, revenue)  
plt.title("Classic Style")  
plt.show()
```



d) Comparison of Styles (2 Marks)

Style	Appearance	Advantage
Ggplot	Grid background	Easy to read trends
Seaborn	Smooth & modern	Attractive and professional
Classic	Simple	Clear and minimal

e) Most Suitable Style (2 Marks)

Seaborn Style

Justification

- Provides **clean and modern look**
- Better for **financial reports & presentations**
- Improves **clarity and aesthetics**